Primoji: Compositional Semantic Tokenization for Language Model Training

Frane Bandov
offbyte
frane@offbyte.com

Abstract

We introduce Primoji, a compositional semantic tokenizer that replaces BPE's frequency-driven subword vocabulary with a semantically structured vocabulary built from Unicode emoji, Natural Semantic Metalanguage (NSM) primitives, common word tokens, and byte fallback. Unlike BPE, which discovers subword units from corpus statistics, Primoji constructs token meanings compositionally: complex concepts are expressed as short sequences of semantically grounded primitives (e.g. `[PLANT, HAVE, LIGHT]` = photosynthesis, `[WATER, CAUSE, AIR]` = evaporation). The resulting vocabulary has roughly 10,000 tokens, three times smaller than standard BPE's 32,768.

We demonstrate that compositional semantic tokenization can achieve performance comparable to BPE on FineWeb-Edu while using a 3× smaller vocabulary. At 125M parameters, Primoji achieves 5.5% lower bits-per-byte (BPB) than BPE at equal FLOPs (1.080 vs. 1.144 at one epoch). At 1B parameters, the two approaches converge to near-parity, with Primoji winning by 2% per document and tying at equal FLOPs. The contribution is not the BPB margin at any particular scale, but the demonstration that a fundamentally different and 3× smaller vocabulary built from semantic primitives is competitive with BPE's 32,768-token frequency-driven vocabulary. We propose research directions enabled by this vocabulary efficiency in Section 5.11, while acknowledging that systematic empirical evaluation of those directions remains future work. We additionally introduce *compositional embedding aliases*, a technique where grammar word embeddings are computed from semantic primitive components (e.g. embedding("is") = mean(BE, NOW)), applicable to any tokenizer. We resolve a byte-preference problem in compositional generation by tracing it to vocabulary gaps rather than architectural limitations. Total experiment cost: under $200 on consumer and cloud GPUs.

1 Introduction

All major language models use Byte Pair Encoding (BPE) or its variants [17, 11]. BPE discovers subword units by iteratively merging frequent byte pairs in a corpus. This produces efficient compression (common words become single tokens), but the resulting vocabulary has no semantic structure. The token "un" appears in both "unhappy" and "university" with no compositional relationship. Empirical analysis attributes BPE's effectiveness primarily to compression efficiency (shorter token sequences) rather than to meaningful subword structure [4].

Human writing systems have explored compositional semantics for millennia. Egyptian hieroglyphs used determinatives, semantic classifiers appended to words to indicate their conceptual category. Chinese characters compose productively through radicals (fire + mountain = volcano; electric + brain = computer). German compounds are fully productive: *Handschuh* (hand + shoe = glove), *Kühlschrank* (cool + cabinet = refrigerator), and novel compounds are routinely coined and understood. A German speaker encountering *Mondschuh* (moon + shoe) immediately grasps its

meaning through decomposition. These systems demonstrate that compositional semantic structure supports productive generalization in human language processing.

Primoji operationalizes this principle for neural language models. We ask: can a fundamentally different vocabulary, built from semantic primitives rather than corpus statistics, achieve performance comparable to BPE? Wierzbicka's Natural Semantic Metalanguage [20, 5] identifies 65 semantic primes, irreducible concepts verified across 30+ language families, that form a complete basis for defining word meanings. Primoji grounds its compositional vocabulary in these primes, supplemented with domain expansions and direct tokens for high-frequency vocabulary.

Our results demonstrate that compositional semantic tokenization can match BPE's training efficiency with a 3× smaller vocabulary. At 125M parameters, Primoji achieves 5.5% lower BPB than BPE at equal FLOPs (1.080 vs. 1.144). At 1B parameters, the two approaches converge to near-parity at equal FLOPs. The central contribution is not the BPB margin at any particular scale, but the demonstration that a 3× smaller vocabulary built from semantic primitives is competitive with BPE's 32,768-token frequency-driven vocabulary. We propose research directions enabled by this vocabulary efficiency in Section 5.11, while acknowledging that systematic empirical evaluation of those directions remains future work.

Status of this work. This is an exploratory empirical preprint. The architectural and applicational directions discussed in Section 5.11 remain hypotheses motivated by the vocabulary efficiency we demonstrate, not empirical findings.

2 Background and Motivation

2.1 Limitations of BPE

BPE [17] and its variants [23, 10] define vocabularies through corpus statistics. This has several consequences relevant to our work.

Fragmentation of semantically related forms. BPE may tokenize "run", "running", "runner" into unrelated subword sequences, forcing the model to independently learn their shared semantics. MorphPiece [8] showed that morphologically aware tokenization improves LLM training across benchmarks, demonstrating that linguistic structure in tokenization is beneficial. MorphBPE [1] confirmed these gains at both 300M and 1B parameter scales across four languages.

Catastrophic digit handling. BPE merges common digit sequences (e.g. "2024") into single tokens while splitting unfamiliar ones unpredictably. Singh and Strouse [18] demonstrated that single-digit tokenization improves math accuracy by +22 percentage points.

Limited compositionality. While BPE subwords are often presented as compositional, controlled experiments show that frequency alone accounts for the majority of BPE's effectiveness [4]. BPE's compositionality contributes only marginally to model performance.

2.2 Natural Semantic Metalanguage

Wierzbicka [20] and Goddard and Wierzbicka [5] identified 65 semantic primes: concepts that cannot be decomposed further and that exist in all studied human languages (verified across 30+ language families over five decades of cross-linguistic research). Examples include: SOMEONE, SOMETHING, DO, HAPPEN, GOOD, BAD, BIG, SMALL, THINK, KNOW, WANT, SAY, MOVE, THERE_IS.

These primes form a complete basis for defining word meanings: the Longman Defining Vocabulary (2,000 words) defines all 80,000+ dictionary entries using only basic concepts at this level of granularity. DeepNSM [2] recently fine-tuned LLMs to generate NSM explications (decomposing words into their primitive meanings), using NSM as an output format. Our approach is complementary: we use NSM primitives as input tokens, embedding the semantic structure directly into the vocabulary rather than training models to produce it.

2.3 Emoji as Semantic Tokens

Unicode defines 1,388 base emoji, of which 1,182 remain after deduplication (stripping U+FE0F variation selectors and removing overlaps with primitive emoji). These provide a ready-made pictographic vocabulary validated by several resources: Unicode CLDR v48 annotations, EmojiNet [22], emoji2vec [3], and ELCo [24]. The ELCo dataset demonstrated that humans can compose emoji to express complex phrases with 80%+ accuracy, validating emoji composition as a viable encoding strategy.

3 Primoji Tokenizer Design

3.1 Vocabulary Structure

The vocabulary is organized into four tiers with strict encoding precedence.

Tier 1a: Direct Emoji (∼1,182 tokens). All Unicode emoji after deduplication (variation selectors stripped before comparison, removing 18 hidden duplicates). Each maps to one or more English words via the dictionary (Section 3.2).

Tier 1b: Common Word Tokens (∼1,276 initially, expandable to ∼7,800 with corpus-frequency additions). High-frequency English words from the New General Service List that have no emoji equivalent. These receive direct single-token encoding, avoiding unnecessary primitive composition for common vocabulary. Examples: "government", "important", "different", "however". The word list is expanded by adding high-frequency words from the target training corpus (Section 4.6).

Tier 2: Compositional Primitives (140 tokens). 65 canonical NSM primes [5] plus 75 domain expansions. The expansions cover perceptual primitives (HOT, COLD, HEAVY, LINE) following the NSM tradition of perceptual molecules, grammatical markers (WITH, FOR, ABOUT), domain terms (ENERGY, COLOR, HEALTH, ELECTRIC, STUDY), emotions (LOVE, FEAR), and environmental and scientific concepts (ENVIRONMENT, SUBSTANCE, VISIBLE, BODY_PART, DEGREE).

Tier 3: Structural and Geographic (∼830 tokens). 258 country flags as geographic modifiers in compositions, ∼500 proper noun anchors extracted from 100K FineWeb-Edu documents via spaCy NER, and ∼72 structural tokens (punctuation, typography, Greek letters, math symbols, currency signs).

Tier 4: Byte Fallback (258 tokens). Following the Llama tokenizer pattern, any byte sequence not captured by Tiers 1–3 is encoded as individual UTF-8 bytes wrapped in boundary markers. This guarantees zero UNK tokens. Every input can be encoded.

Total vocabulary: 4,035 tokens in the initial V1 configuration, growing to 5,311 by V3 (still 6× smaller than BPE's 32,768), and expandable to roughly 10,200 with corpus-frequency word tokens (3× smaller than BPE).

A note on emoji and naming. The name Primoji (primitives + emoji) reflects the project's origin, but emoji are not central to the contribution. At the model level, all tokens are integer IDs. The model never sees emoji glyphs, and there is no technical dependence on Unicode emoji. An emoji token, a word token, and a primitive token are mechanically identical: integers looked up in an embedding table. The distinction between "emoji" and "concrete noun" in our generation analysis is purely a matter of which ID range a prediction falls into. Any concept not represented by an existing emoji can be added as a word token or composed from primitives. The emoji tier is a convenient source of ∼1,200 visually distinct tokens for concrete nouns and a useful notation for human-readable examples in this paper, but it could be replaced entirely by word tokens without changing the architecture, the training procedure, or the results. In generated output, emoji tokens are decoded to their English words ("dog", "house", "sun"), not to emoji glyphs.

3.2 Dictionary and Encoding Pipeline

The encoding pipeline has five stages with strict precedence:

1. **Exact lookup.** If the input word matches a dictionary entry (emoji, word token, anchor, or pre-computed composition), emit the corresponding token(s).
2. **Contraction expansion.** Contractions are split at the apostrophe before encoding. "Won't" becomes "will" + "not", "don't" becomes "do" + "not". With compositional embedding aliases (Section 4.4), "not" is a word token whose embedding is computed from `[NOT]`, and "will" is a word token whose embedding is computed from `[AFTER]`. This unifies negation across surface forms.
3. **Conservative SymSpell.** Edit distance 1, unique candidate only, 4+ character words only. Aggressive spell correction is known to introduce 12–16% false positives; noisy-trained models outperform clean-trained models on real input.
4. **Auto-composition.** OOV words are composed from primitives using WordNet-based decomposition (∼4,500 auto-generated compositions). Example: "metabolic" → `[MATTER, CHANGE]`, "researchers" → `[STUDY, SOMEONE]`.
5. **UTF-8 byte fallback.** Remaining characters are encoded as individual bytes.

Dictionary construction. The dictionary maps English words to emoji and primitive compositions. It was built from layered sources: Unicode CLDR v48 annotations, emojilib community keywords, EmojiNet sense mappings [22], emoji2vec embedding pairs [3], and ELCo compositional mappings [24]. Roughly 20K dictionary entries total.

Canonical decode rule. For the reverse lookup (IDs to word), priority is: single-word CLDR name > corpus frequency (via the wordfreq library) > shortest form. Multi-word CLDR names (e.g. "face with tears of joy") never beat single-word forms (e.g. "joy").

Table 1: Compression progression of the Primoji tokenizer on 1,000 FineWeb-Edu sentences. Ratio is Primoji tokens divided by BPE tokens (lower is better). The byte-fallback column reports the percent of word occurrences that fall through to the UTF-8 byte tier.

Version	Vocab	Tokens (Primoji)	Ratio vs BPE	Byte fallback
v1	4,035	59,460	1.795×	17.4%
v2	4,081	58,872	1.778×	16.6%
v3	5,311	57,120	1.725×	16.6%
v4	18,464	50,587	1.527×	13.6%
v7	10,195	36,076	**1.089**×	**6.9%**
BPE (Mistral)	32,768	33,118	1.000×	0%

Decoder architecture. The decoder uses the vocabulary directly (catalog → primitives → anchors → bytes). The dictionary is encode-only. The decoder does not use dictionary reverse lookup. This was a design correction: reverse lookup created ambiguity because multiple words map to the same composition.

Seed file format. The dictionary seed stores symbolic references (emoji characters, primitive names), not numeric IDs, so they survive vocabulary re-sorts. The dictionary build pipeline is reproducible from a single script.

3.3 Composition Examples

Example compositions: `[PLANT, HAVE, LIGHT]` = photosynthesis; `[WATER, CAUSE, AIR]` = evaporation; `[HEALTH, LIVE, SMALL]` = infection; `[ELECTRIC, ENERGY]` = electricity; `[STUDY, SOMEONE]` = researchers. Country flags serve as geographic modifiers (258 flags = 258 geographic contexts). Proper nouns: top ∼500 in FineWeb-Edu receive dedicated anchor tokens; the remainder compose with graceful lossiness.

3.4 Math and Code Handling

Math and code are not composed through semantic primitives. Single-digit tokenization is used for digits, following Singh and Strouse [18] which showed +22pp math accuracy improvement. Math operators (+, −, ×, ÷, =, ≈, ≤, ≥, π, ∫, Σ, etc.) receive atomic tokens. Code keywords get dedicated tokens.

4 Evaluation

4.1 Coverage and Compression

We evaluate on 1,000 FineWeb-Edu sentences [14], comparing against the Mistral v0.3 BPE tokenizer (vocabulary size 32,768).

The compression ratio improved through iterative vocabulary expansion. Initial versions of Primoji produced 1.7–1.8× as many tokens as BPE; the V7 configuration with 10K vocabulary reaches 1.09× at 6.9% byte fallback. Table 1 reports the progression and Figure 1 visualises it.

The improvement from 1.79× to 1.41× came primarily from auto-composing ∼4,500 OOV words from primitives, replacing expensive byte sequences with 2–3 primitive compositions. The

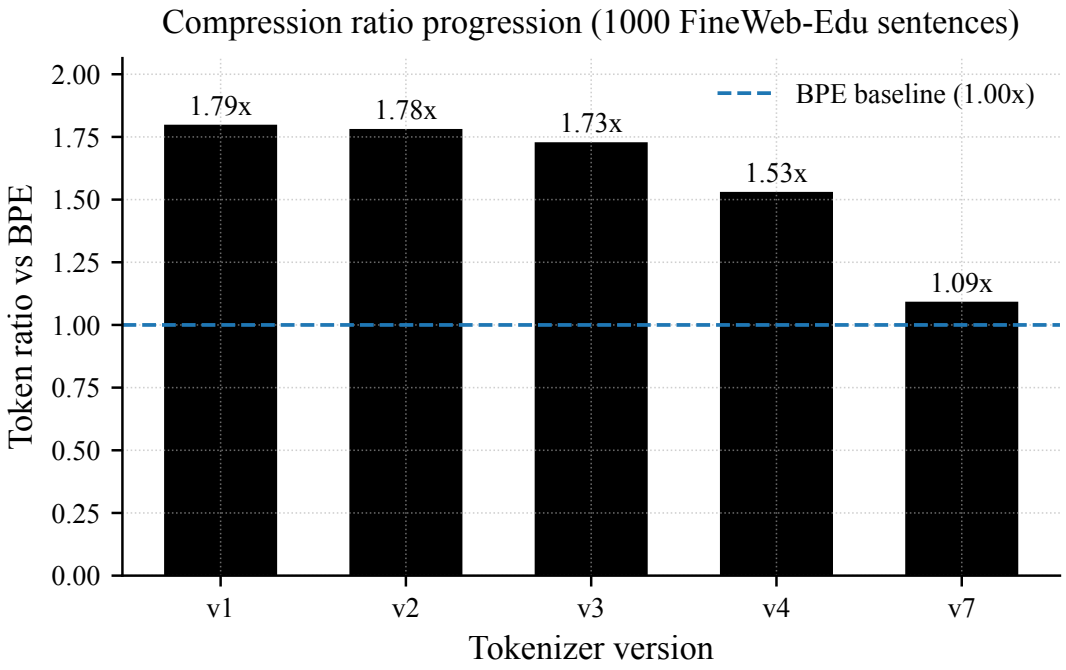

Figure 1: Compression ratio across Primoji tokenizer versions. The dashed line is BPE parity. The improvement from 1.79× (v1) to 1.09× (v7) came from auto-composing OOV words and adding high-frequency words from FineWeb-Edu to the word-token tier.

improvement from 1.41× to 1.09× came from adding ∼5,000 high-frequency words from FineWeb-Edu training data as direct word tokens, and from splitting hyphenated words. The compression "failure" was a vocabulary gap, not a fundamental limitation of compositional tokenization. The median sentence is now at exact BPE parity (1.00×) with a vocabulary still 3× smaller than BPE.

4.2 Training Setup

We conduct controlled training experiments comparing Primoji and BPE on identical FineWeb-Edu documents.

Setup. Two transformers with identical architecture and hyperparameters, differing only in vocabulary size. Both models see the same documents in the same order. The full configuration is given in Table 2.

Metric. Bits-per-byte (BPB) is the sole comparison metric. BPB normalizes cross-entropy loss by the original text byte count, making it independent of vocabulary size and sequence length:

$$\text{BPB} = \frac{\sum \text{CE loss (nats)}}{\text{original text bytes} \times \ln(2)}.$$

This is the same metric used by Pagnoni et al. [13] for cross-tokenizer comparison. Validation BPB is computed on a held-out 10% split of the training corpus, using 100 random validation batches per evaluation.

Table 2: Training configuration. The 125M and 1B columns differ only in architecture size, learning rate, and effective batch composition; all other settings are shared. Both Primoji and BPE runs use the identical configuration in their respective size column.

	125M	1B
Layers	12	24
Hidden dim	768	2048
Attention heads	12	16
FFN dim	3072	5461
Activation	SwiGLU	SwiGLU
Normalization	RMSNorm	RMSNorm
Sequence length	1024	1024
Effective batch (tokens)	$32 \times 1024 = 32{,}768$	$32 \times 1024 = 32{,}768$
Micro batch / grad accum	32×1	4×8
Optimizer	AdamW	AdamW
β_1, β_2	$0.9, 0.95$	$0.9, 0.95$
Weight decay	0.1	0.1
Gradient clip	1.0	1.0
Peak LR	6×10^{-4}	3×10^{-4}
Min LR	6×10^{-5}	3×10^{-5}
LR schedule	cosine	cosine
Warmup steps	2,000	2,000
Mixed precision	bf16	bf16
Eval interval	every 200 steps	every 500 steps

BPB calculation note. An earlier version of our pipeline contained a bug dividing by total validation bytes rather than evaluation-batch bytes, producing unrealistically low values (0.08 instead of 1.33). All reported values use the corrected formula. Tests were added to prevent regression.

Reproducibility. The reference implementation for the tokenizer, the dictionary build pipeline, and the training loop is available at `https://github.com/frane/primoji`. Concretely:

- `scripts/build_dictionary.py` reproduces the symbolic dictionary seed from primitive synonyms, layered emoji catalogs, NER anchors, and WordNet auto-compositions. Layer precedence is fixed: primitive synonyms override emoji catalog entries, and word tokens override single-primitive compositions to preserve reverse lookup.
- `scripts/prepare_training_data.py` streams FineWeb-Edu documents and writes binary token shards plus per-token tier IDs.
- `scripts/train.py` runs the experiments described in this section using the configuration in Table 2, switchable between the 125M and 1B preset via `--model-size`.

The token ID layout is frozen as part of the package's public contract: emoji occupy IDs 0 through (emoji catalog size) − 1, the 140 primitives occupy 1200 through 1339, country flags follow, then word tokens, anchors, and structural tokens, with the 258 byte-fallback tokens at the end of the vocabulary.

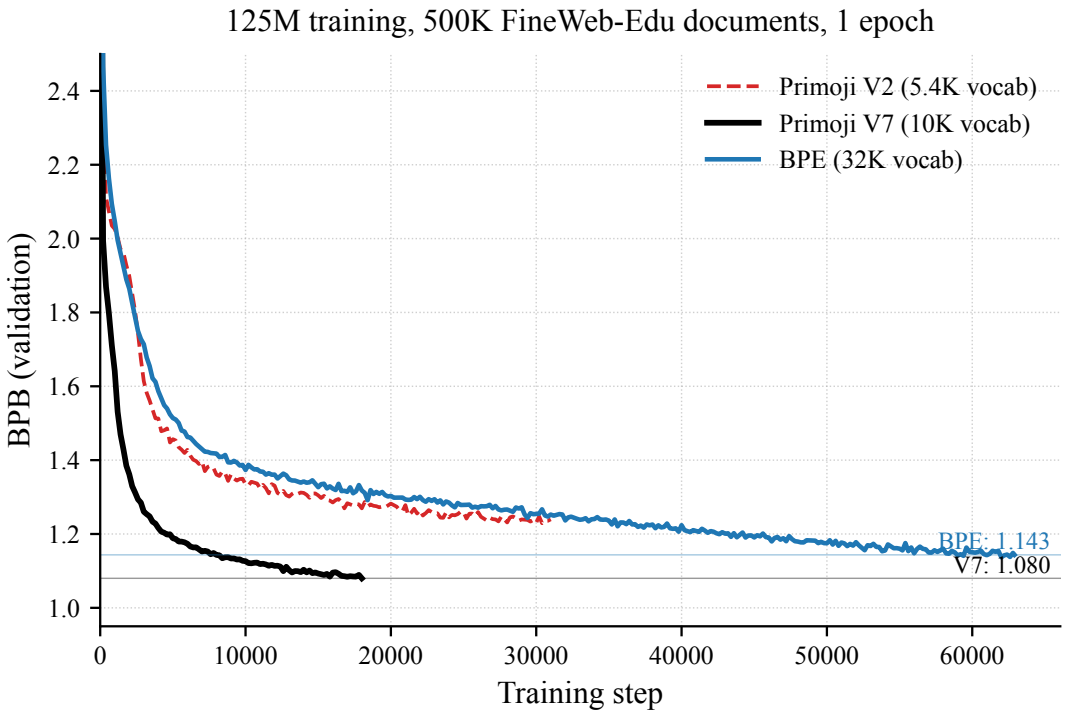

Figure 2: Validation BPB during 125M training, 500K FineWeb-Edu documents, 1 epoch. Primoji V7 (10K vocab) reaches 1.080 BPB; BPE (32K vocab) reaches 1.144. Both models use identical architecture (12 layers, 768 hidden) and identical hyperparameters.

4.3 125M Training Experiment

We trained matched 125M models on 500K FineWeb-Edu documents for one epoch each. Figure 2 shows the validation BPB throughout training. Primoji V7 reaches 1.080 BPB, a 5.5% improvement over BPE's 1.144. The advantage is present from approximately 5% training progress onwards and never reverses within the epoch.

Equal-FLOP comparison. A central observation is that primoji and BPE use nearly identical compute at one epoch despite different token counts. Primoji processes ∼590M tokens with 122M parameters; BPE processes ∼515M tokens with 139M parameters. Approximating training FLOPs as $6 \times \text{params} \times \text{tokens}$, both models perform roughly 4.3×10^{17} FLOPs. Primoji's 14% longer sequences are offset by its 12% fewer parameters, because primoji's 10K embedding table is three times smaller than BPE's 32K embedding table. At equal FLOPs, equal training time, and equal cost, primoji achieves 5.5% better BPB. Figure 3 visualizes this for both 125M and 1B.

Parameter reallocation. The smaller embedding table has a secondary effect: it reallocates parameters from lookup to computation. At 125M, BPE spends 18% of its parameter budget on embeddings versus primoji's 6%. At 1B, the 47M difference is roughly 3% of total parameters and the reallocation effect is much smaller. We have not systematically tested architectures that exploit this freed capacity in this work and discuss possible directions in Section 5.11.

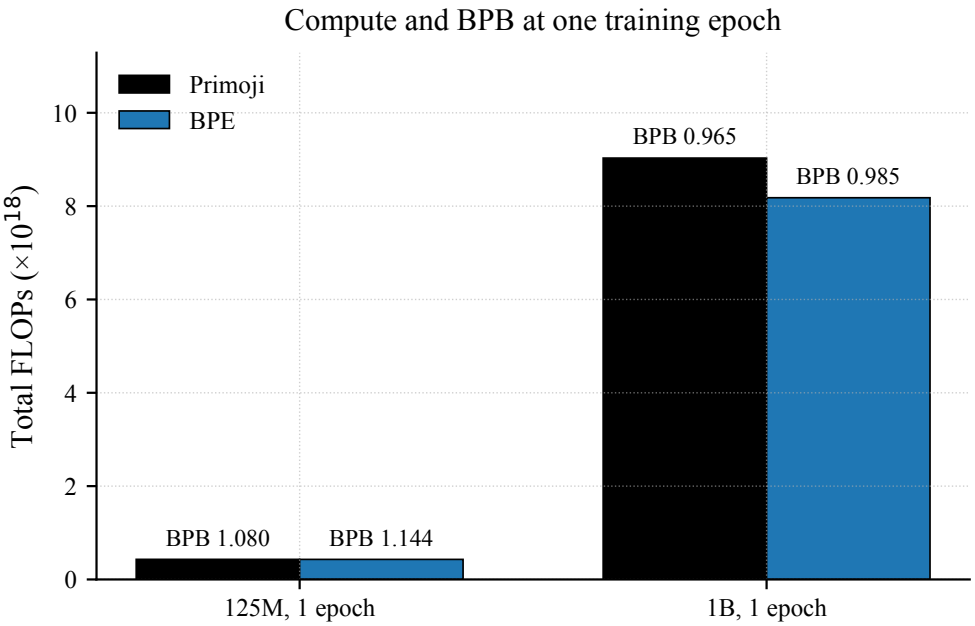

Figure 3: Total training FLOPs and final BPB at one epoch. At 125M, Primoji and BPE are within 0.3% on FLOPs, with Primoji 5.5% lower in BPB. At 1B, Primoji uses 10% more FLOPs (smaller embedding savings as a share of total parameters), and the BPB advantage shrinks from 5.5% to 2% per document.

4.4 Composition-Aware Training Variants

We trained seven variants on the same 500K documents, iteratively addressing the byte-preference problem (Section 4.5), the primitive/word-token boundary, and embedding design. Table 3 summarises the configurations and results.

All Primoji variants beat BPE on BPB. The progression reveals what matters. V1 achieves strong BPB (1.085) but generates almost exclusively word tokens with minimal primitive usage (1.6%). V2's aggressive byte downweighting (0.3×) forces more primitive generation (9.2%) but produces less coherent text and slightly worse BPB. V3 at 0.7× byte weight provides a better balance (8.6% primitives, BPB 1.092). V4 combines the 0.7× byte weight and tier embeddings with an expanded vocabulary of 18,464 total tokens, the additional slots filled with high-frequency English words from the wordfreq library on top of the V3 base of 5,311; it matches V1's BPB (1.085) while processing 36% fewer tokens than V3 on the same documents.

Primitive ID conflict and V5. V4 training revealed that 126 of 140 primitive words ("good", "bad", "think", "know") were overridden by Tier 1b word tokens during dictionary construction. V5 fixed this. The result: V5 achieves the best single-epoch BPB of the non-alias variants (1.073) and generates 37.3% primitive tokens. The model produces text using semantic primitives as natural building blocks. However, V5 also exposed a tension: mapping grammar words like "is", "not", "this" to word tokens instead of primitives contradicts NSM theory, which identifies these as irreducible semantic concepts. Mapping them to primitives (as in earlier versions) degraded readability.

The resolution: compositional embedding aliases (V6). Grammar words remain as single tokens (preserving compression and readability), but their embeddings are not independent learned vectors. Instead, each grammar word's embedding is computed as the mean of its primitive components:

Table 3: Tokenizer variants at one epoch on 500K FineWeb-Edu documents, 125M parameters. "Byte wt" is the loss-weight multiplier applied to byte-fallback tokens. "Tier emb" is whether the model receives a learned 5-way tier-type embedding. "Aliases" is the number of grammar words whose embeddings are composed from primitive components (Section 4.4). All BPB values are at one epoch (equal data exposure). Both Primoji and BPE consume approximately equal FLOPs at one epoch ($\sim 4.3 \times 10^{17}$).

Variant	Total vocab	Byte wt	Tier emb	Aliases	BPB	vs BPE
V1	4,035	1.0	no	0	1.085	−5.1%
V2	4,081	0.3	yes	0	1.096	−4.2%
V3	5,311	0.7	yes	0	1.092	−4.5%
V4	18,464	0.7	yes	0	1.085	−5.1%
V5	10,102	0.7	yes	0	1.073	−6.2%
V6	10,149	0.7	yes	80	1.081	−5.5%
V7	10,195	0.7	yes	80	**1.080**	**−5.5%**
BPE	32,768	n/a	n/a	n/a	1.144	baseline

```
embedding("is") = mean(embedding(BE), embedding(NOW))
embedding("was") = mean(embedding(BE), embedding(BEFORE))
embedding("they") = mean(embedding(SOMEONE), embedding(OTHER), embedding(MANY))
embedding("not") = mean(embedding(NOT))
```

At one epoch, V6 achieves 1.081 BPB and V7 achieves 1.080 BPB, both a 5.5% improvement over BPE's 1.144. V6 and V7 are within noise of each other and within noise of V1 (∼1.08 at one epoch). Within the tested variants and at one epoch, we do not observe evidence that aliases or coverage fixes materially change the main BPB advantage.

V7 adds 46 common English words that were erroneously hitting byte fallback ("business", "humans", "place", "force", "social"). This did not affect BPB but improved inference quality by reducing byte-fallback tokens in generation. Grammar words render as normal English while their embeddings carry the compositional structure of their NSM primitives.

Compositional embedding aliases achieve three things simultaneously: compression stays at ∼1.1× (one token per grammar word); the model sees compositional structure in every embedding (related forms share primitive components); and NSM primes are represented in the embedding space even when they are not explicit tokens in the sequence. Gradient updates to the `[BE]` embedding simultaneously update the embeddings of "is", "was", "are", "were", "been", and "being", enabling compositional generalization across grammatical forms. Alias tokens have zero independent parameters; their representations are entirely determined by their primitive components.

4.5 The Byte-Preference Finding and its Resolution

A critical finding emerged during inference evaluation: with the small initial vocabulary (V1, 4,035 tokens), the trained model generated ∼95% byte-fallback tokens and only ∼5% primitive or word tokens. Rather than composing concepts from primitives, the model had learned to spell words character by character through the byte channel.

This is rational behavior given the training data: 16.6% of training tokens were byte-encoded, and byte sequences are locally predictable (each character follows naturally from English spelling patterns). Predicting the next byte requires choosing among ∼26 likely options. Predicting the next primitive requires understanding the full concept being expressed, a harder task with 140 options and no local predictability cues.

Table 4: 1B comparison at 1 epoch on 1M FineWeb-Edu documents. Per document, Primoji achieves 2% lower BPB. At equal FLOPs the two approaches tie.

Metric	Primoji 1B	BPE 1B
BPB	**0.965**	0.985
Parameters	1.28B	1.32B
Tokens (1 epoch)	1.18B	1.03B
FLOPs (1 epoch)	9.03×10^{18}	8.18×10^{18}
Wall time (1 epoch)	26.7 h	21.4 h

We initially attributed this to a fundamental mismatch between compositional tokens and standard autoregressive training. Three modifications were introduced to address it: aggressive auto-composition reducing byte fallback from 16.6% to 10.5%; tier embeddings to let the model distinguish compositional from byte channels; and byte loss downweighting to scale gradients for byte tokens.

The actual fix proved simpler. Expanding the total vocabulary from V1's 4,035 tokens to V7's 10,195 tokens eliminated byte generation entirely. The byte-preference problem was a vocabulary gap, not an architectural incompatibility. Adequate coverage of common English words removed the model's incentive to fall through to bytes. This reframes the contribution of tier embeddings and byte loss downweighting: they are useful for increasing primitive usage in generation but are not necessary to eliminate the byte-preference problem.

4.6 Vocabulary Expansion

The compression improvement from 1.41× to 1.09× came from two changes: adding ∼5,000 high-frequency words from FineWeb-Edu training data as direct Tier 1b tokens, and splitting hyphenated words ("insulin-deprived" becomes "insulin" + "deprived", both of which encode as direct tokens). Composition fires on approximately 3% of text, concentrated on the long-tail rare and technical vocabulary where the semantic scaffold is most likely to aid learning. V4 training with the expanded vocabulary confirmed that BPB held at 1.085 (matching V1) while processing 36% fewer tokens than V3. The vocabulary expansion improved compression without sacrificing learning efficiency.

4.7 Scaling to 1B Parameters

To test how the BPB advantage scales with model size, we trained matched 1B-parameter models for primoji and BPE on 1M FineWeb-Edu documents. Both models use identical architecture (24 layers, 2048 hidden, 16 heads, 5461 FFN), data, and hyperparameters. This isolates the effect of tokenization from architectural differences.

Per document, Primoji achieves 2% lower BPB. However, primoji used 10% more FLOPs (longer sequences offset by smaller embeddings, but the offset is smaller at 1B than at 125M because the embedding savings are only ∼3% of total parameters at this scale). At equal FLOPs, the two models tie. The 5.5% advantage observed at 125M largely disappears at 1B. We interpret this convergence as expected. At 125M, the 17M-parameter difference between primoji (122M) and BPE (139M) is 14% of the model and the smaller embedding represents a meaningful capacity reallocation. At 1B, the 47M difference is only ∼3% and the reallocation effect is small. Larger models also have enough capacity to extract compositional structure from BPE's unstructured vocabulary on their own, reducing the relative benefit of providing it explicitly.

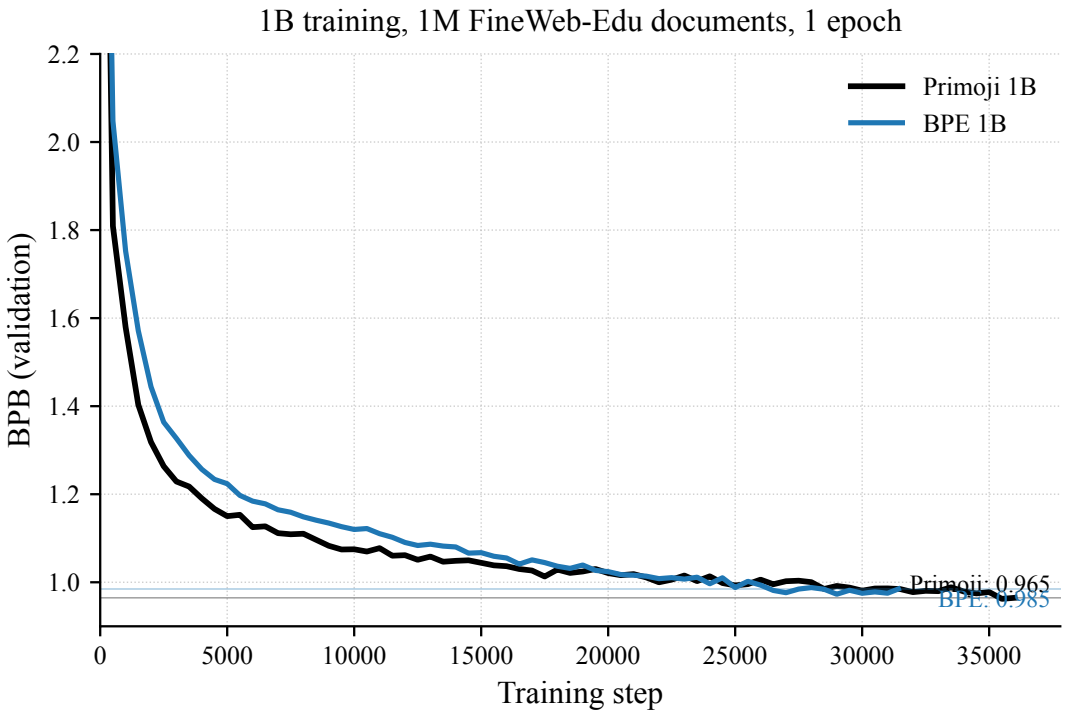

Figure 4: Validation BPB during 1B training, 1M FineWeb-Edu documents, 1 epoch. Primoji ends at 0.965 BPB, BPE at 0.985. The two curves track each other closely throughout training.

Architecture is held constant. Both models use BPE-standard architecture (24 layers, 2048 hidden). This is a deliberate choice to isolate tokenization from architectural variables. Whether primoji-optimized architectures could recover or grow the BPB advantage at scale is an open question. We tested two preliminary architectural variants enabled by primoji's smaller vocabulary and observed only modest gains consistent with parameter scaling laws within standard transformer templates. The architectural directions discussed in Section 5.11 target qualitatively different model classes than these reallocation experiments and remain untested.

Multi-epoch behavior. BPB continues to improve with additional epochs. At 125M, V7 drops from 1.080 (1 epoch) to 0.997 (3 epochs). At 1B, epoch 1 reaches 0.965 and epoch 2 reaches 0.916, with a best of 0.907 at step 73K. All BPE comparisons in this paper use matched 1-epoch values; multi-epoch results are reported only to show continued learning.

4.8 Generation Quality Evaluation

This section presents qualitative analysis of generation behavior, not a rigorous evaluation of language quality. We do not perform human ratings or task-based metrics; these are left to future work.

To evaluate generation quality beyond BPB, we generated text from the V7 125M model on 200 diverse prompts spanning science, history, ethics, technology, and everyday topics. We measure the distribution of generated tokens across tiers using semantic units (grouping multi-token byte sequences into single word units) rather than raw token counts, which would inflate byte-fallback statistics by 8–10× per OOV word.

Table 5 reports the tier distribution and Figure 5 visualises it.

Table 5: Tier distribution of generated tokens for V7 125M, 200 prompts. Counts are semantic units; raw tokens are 19,154 due to multi-token byte sequences. Confidence intervals are 95% binomial.

Tier	Semantic units	Percent	95% CI
Word	7,339	54.6%	±6.7%
Structural	2,580	19.2%	±5.3%
Primitive	1,849	13.7%	±4.6%
Emoji	885	6.6%	±3.3%
Byte fallback	799	5.9%	±3.2%

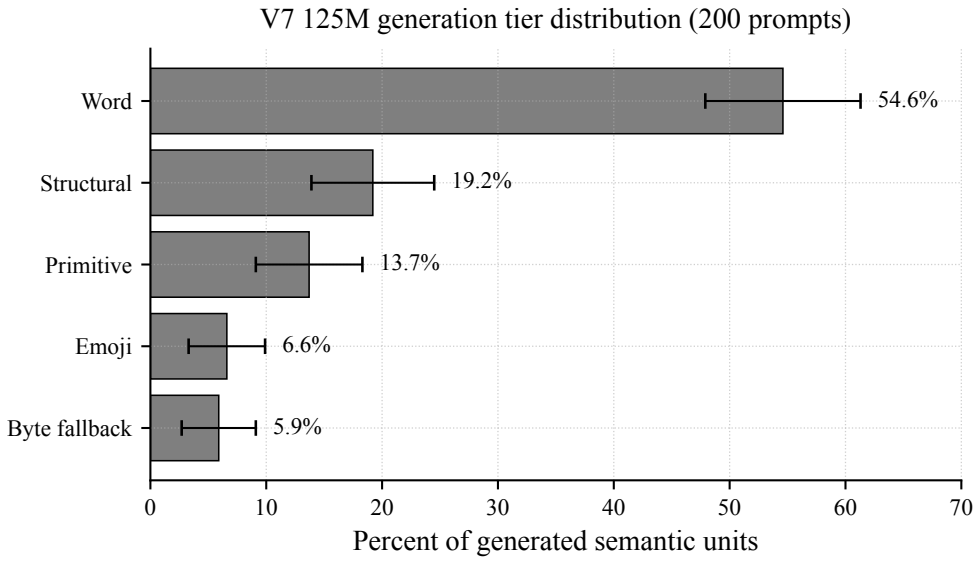

Figure 5: Generation tier distribution for V7 125M, 200 prompts. Words dominate at 55%; primitives account for 14% of output; byte fallback is limited to 6% of conceptual content.

The model generates coherent, on-topic text with a natural mix of English words and semantic primitives. Words dominate at 55%, rendering grammar and common vocabulary as readable English. Primitives account for 14% of output, with content concepts (SOMEONE, LIVE, EARTH, MOVE, THINK, PLANT) appearing naturally in generated text. Emoji tokens contribute 7%, providing concrete nouns. Byte fallback at 6% is limited to proper nouns and rare terms.

Representative samples from the V7 125M model:

> "photosynthesis is process which plants convert light energy"
>
> "earthquake is caused move materials earth"
>
> "democracy is system which means power, consent, decency"

The model demonstrates compositional understanding: "earthquake" was never a single token during training, but the model generates its component concepts correctly using `[MOVE]`, `[MATTER]`, `[EARTH]` primitives.

The 13,452 semantic units correspond to 19,154 raw tokens, a 42% inflation caused by byte-fallback words each consuming 8–10 raw tokens. This distinction between raw tokens and semantic units is methodologically important: raw token counts make byte fallback appear dominant when it is in fact limited to 6% of the conceptual content.

5 Discussion

5.1 Primoji as Semantic Generative Code

Primoji is not merely a tokenizer but a semantic generative code. Each composition encodes a meaning, not just a string. This has precise implications. Known compositions reconstruct exactly: if `[PLANT, HAVE, LIGHT]` maps to "photosynthesis" in the dictionary, decoding is unambiguous and lossless. Novel compositions are lexically underdetermined: a model generating `[ANIMAL, HOME, FR]` could mean "French bulldog", "Parisian stray", or "pet in France". The composition encodes semantic content but not the exact surface form. The compositional output space is larger than the lexical space of English. This distinction (lossless for known vocabulary, approximate for novel compositions) is the defining characteristic of Primoji. It is closer to a semantic representation layer with byte-safe fallback than to a standard tokenizer.

5.2 Dual-Head Training

To address lexical underdeterminacy, we propose dual-head training. A *primary head* predicts the next Primoji token (standard autoregressive objective). An *auxiliary head* ($\lambda = 0.1$) predicts the original surface word at composition boundaries ($\sim$20K dictionary classes). This creates a "semantic primary channel + surface auxiliary channel" architecture. The primary channel provides compositional structure for learning; the auxiliary channel preserves surface realization for generation. No direct prior work exists on this factored representation approach for tokenization.

5.3 Composition-Level Prediction

The byte-preference finding (Section 4.5) suggests that standard autoregressive prediction at the individual token level may be fundamentally mismatched with compositional structure. An alternative architecture would predict at the composition level: a local encoder pools primitive tokens within each composition into a single concept vector; a global transformer predicts the next concept (standard autoregressive, but over compositions rather than tokens); and a local decoder expands each concept back to its primitive sequence. This parallels Meta's Byte Latent Transformer [13], which processes byte-level input through entropy-based patches. In Primoji's case, patch boundaries are determined by semantic composition boundaries rather than entropy.

5.4 Compositional Embedding Aliases

The V5 experiment exposed a design tension rooted in NSM theory. Wierzbicka's 65 primes explicitly include NOT, IF, BECAUSE, I, YOU, THIS, ALL, SOME. In NSM these are irreducible semantic concepts, not merely grammar words. Mapping them to word tokens contradicts the theoretical foundation. Mapping them to primitive tokens produces the best BPB (1.073) but degrades readability.

We resolve this tension with compositional embedding aliases. Grammar words remain as single tokens in the sequence (preserving compression and surface readability), but their embeddings are not independently learned. Instead, each alias token's embedding is computed as the mean of its primitive component embeddings.

This has several properties. First, related grammatical forms share primitive components: "is", "was", "are", "were" all share the `[BE]` embedding. A gradient update to `[BE]` simultaneously updates all related forms, enabling compositional generalization across the grammar. Second, alias

tokens have zero independent parameters. Third, the sequence length is unaffected: "is" remains one token.

At one epoch (equal data exposure), V6 with aliases achieves 1.081 BPB, essentially identical to V1 without aliases (∼1.08 BPB). The alias mechanism does not improve BPB at this scale and training duration. The 5.5% advantage over BPE comes from the compositional tokenization itself. Aliases remain architecturally novel and may show benefits at larger scale or in domain-specific settings.

The technique is not specific to Primoji. Any tokenizer could use compositional aliases by defining a mapping from tokens to semantic or morphological components. Concurrent work on vocabulary compression [15] explores a related idea: replacing surface-form embeddings with compositions of base-form embeddings and learned transformation vectors. Our approach differs in three respects: our components are semantic primitives grounded in linguistic theory, not learned morphological offsets; our aliases share primitive embeddings across semantically related forms, not just morphologically related ones ("was" and "before" share `[BEFORE]`); and our empirical result is stronger (5.5% BPB improvement at equal FLOPs versus modest gains with some performance loss). The convergent exploration of compositional embedding structures from independent directions suggests this is a promising research area.

5.5 Primoji as a Configurable Tokenization Framework

A core contribution of this work is not a single tokenizer but a tokenization framework with multiple tunable parameters. BPE offers one dial: vocabulary size (number of merge operations). Primoji exposes at least five independent design dimensions, summarised in Table 6.

Table 6: Configurable dimensions of the Primoji tokenization framework.

Dimension	Description
Vocabulary size	Controls how many words get direct single-token encoding versus compositional encoding. From ∼2,400 tokens (∼30% composed) to ∼18,000 tokens (∼2% composed).
Composition depth	Maximum number of primitives per concept. Depth 3 produces coarse decompositions; depth 5 produces precise ones at the cost of longer sequences.
Primitive count	Granularity of the compositional vocabulary. Adding domain-specific primitives creates finer distinctions and shorter compositions for that domain.
Composition rate	Fraction of tokens that are primitives versus direct words, determining the strength of the semantic-structure signal.
Domain specialization	The primitive set, word list, and composition dictionary are all swappable without changing the architecture.

These dimensions interact. At 2,400 tokens, ∼30% of text is composed from primitives, giving maximum semantic structure and maximum sequence expansion (1.7×). At 10,400 tokens, ∼3% composes with near-BPE compression. No other tokenization approach offers this range. BPE, WordPiece, and Unigram LM produce a single vocabulary from corpus statistics. Byte-level models [13] have no vocabulary design choices at all. Primoji allows practitioners to explicitly trade compression for semantic structure based on their compute budget, domain, and task requirements.

5.6 The Design Challenge: Vocabulary as Engineering

The configurability of Primoji is both its strength and its primary engineering challenge. Unlike BPE, where a single algorithm produces the vocabulary automatically from corpus statistics, Primoji requires deliberate design decisions at every level. Our experimental progression from V1 to V7 illustrates how these decisions impact outcomes.

Where to draw the primitive/word boundary. Should "is" be a word token or the `[BE]` primitive? In V5, mapping all grammar words to primitives produced the best BPB (1.073) but degraded readability. In V4, keeping them as word tokens preserved readability but disconnected the grammar from the semantic structure. V6's compositional embedding aliases resolved this tension, but finding that solution required five experimental iterations. The general principle we arrived at: content words whose primary meaning is the concept map to primitives ("water" = WATER, "think" = THINK); grammar words map to word tokens but carry compositional structure in their embeddings.

Polysemy and ambiguity. "Lands" can be the plural noun (territories) or the verb (an aircraft lands). Both senses connect to `[EARTH]`, and the model learns to distinguish them from context. Other words resist clean decomposition. "Degree" covers extent, temperature, and academic qualification. We map it to the `[DEGREE]` primitive and let all senses share the compositional link. This is a design choice that could be wrong for some applications, and there is no principled algorithm to make it automatically. BPE avoids this problem entirely by treating words as opaque strings. Quantitative evaluation of polysemy handling on benchmarks such as Word-in-Context is future work.

Vocabulary coverage. Our compression ratio improved from 1.79× to 1.09× across seven versions, primarily by fixing vocabulary gaps. Words like "propagation", "susceptible", and "business" were falling to byte encoding not because they are rare, but because the word list was derived from general English frequency data rather than from FineWeb-Edu. The systematic fix is straightforward (any word appearing N+ times in training data that hits byte fallback should be a word token), but we arrived at it only after multiple rounds of manual patching. Future implementations should build the word list from the target corpus.

The cost of design versus automation. BPE's advantage is that it requires no linguistic knowledge and no design decisions. Primoji's advantage is that it exposes design decisions that can be tuned for specific domains. The tradeoff is real: building a Primoji vocabulary for a new domain requires understanding the domain's conceptual structure, not just running a compression algorithm on its text. Whether this engineering cost is justified depends on the application.

5.7 Honest Assessment of Claims

Established facts. Primoji produces 1.09× more tokens than BPE at ∼10K vocabulary, with vocabulary 3× smaller than BPE (10K vs 32K) at near-parity compression. At 125M parameters (one epoch, 500K documents), Primoji achieves 5.5% lower BPB than BPE (1.080 vs 1.144) at equal FLOPs. At 1B parameters (one epoch, 1M documents), Primoji achieves 2% lower BPB per document (0.965 vs 0.985) and ties at equal FLOPs. Within the tested variants and at one epoch, we do not observe evidence that aliases, tier embeddings, or vocabulary size variations materially

change the main BPB advantage. The byte-preference problem was caused by vocabulary gaps, not architectural incompatibility.

Interpretation. Primoji's BPB advantage over BPE shrinks with scale. At 125M, the 17M parameter difference between primoji (122M) and BPE (139M) is 14% of the model and the smaller embedding is a meaningful capacity reallocation. At 1B, the 47M difference is only ∼3% of the model. The contribution of this paper is not "Primoji beats BPE at all scales" but "Primoji matches BPE at the scales we tested with a vocabulary 3× smaller."

Open questions. Whether primoji-optimized architectures recover or grow the BPB advantage at large scale; whether the BPB parity at 1B holds at 7B and beyond; whether the architectural directions enabled by the smaller vocabulary (Section 5.11) translate to practical advantages; whether compositional embedding aliases show benefits at larger scale or with extended training; whether dual-head training resolves lexical underdeterminacy; and whether composition-level prediction is required for fully compositional generation at inference. We did not run multiple training seeds. Numerical differences smaller than ∼0.01 BPB between primoji variants should be interpreted as within experimental noise rather than meaningful differences.

5.8 Where Primoji May Not Help

While our results demonstrate a 5.5% advantage on educational text, Primoji's compositional design may be less beneficial in some application domains. Verbatim reconstruction is lossy at the word level. Creative and stylistic writing depends on the specific surface form chosen, and BPE preserves "melancholy" versus "sadness" versus "gloom" as distinct tokens; Primoji may collapse them into the same composition. Code generation has rigid syntax that does not decompose into semantic primitives. Conversational and casual text resists compositional decomposition because slang, idioms, and discourse markers carry pragmatic meaning that primitives cannot capture. Factual recall of named entities requires exact tokens, and the long tail composes lossily or falls to byte encoding.

5.9 Limitations

The current dictionary and evaluation use English text. The NSM primes are verified across 30+ languages, and the flag plus primitive composition scheme extends naturally to loanwords, but systematic multilingual evaluation is future work. The choice of primitives imposes a particular semantic decomposition; we mitigate this through configurable domain-specific primitive sets. Our primary results are at 125M and 1B parameters. Whether the 5.5% BPB advantage at 125M and the convergence at 1B continue beyond these scales remains to be confirmed. We do not perform spell correction on training data.

5.10 From Tokenizer to Interlingua

A consequence of compositional tokenization is the gap between the model's operating space (semantic tokens) and human-readable output (surface text). We identify three approaches to surface realization, forming a natural progression.

Stage 1: dictionary lookup (current implementation). For compositions that appear in the dictionary, decoding is exact and deterministic. For novel compositions, decoding is approximate. This is sufficient for evaluating training efficiency via BPB but not for production text generation.

Stage 2: dual-head training. A single model with two output heads, one predicting the next semantic token and one predicting the original surface word. At inference, the surface head generates human-readable text while the semantic head drives the model's internal reasoning. This adds ∼10% parameter overhead and preserves end-to-end trainability.

Stage 3: semantic interlingua with surface transformer. A dedicated "concept model" operates entirely in primoji space, predicting sequences of compositions. A separate lightweight "surface model" translates primoji sequences to fluent natural language. This is architecturally equivalent to neural machine translation from a semantic code to a natural language. The NSM foundation, verified across 30+ language families, provides theoretical grounding for this direction.

We present Stage 1 results in this paper and propose Stage 2 as an immediate extension. Stage 3 is a longer-term research direction with implications beyond tokenization.

5.11 Research Directions

The central contribution of this work is demonstrating that a 3× smaller vocabulary can achieve performance comparable to BPE. We propose that this vocabulary efficiency may enable architectural and applicational directions whose computational cost scales with vocabulary size, though we have not systematically tested these in this work. Many of these possibilities are computationally infeasible with BPE's 32K vocabulary but may become tractable with Primoji's 10K. We outline them as motivation for future work.

Vocabulary-quadratic architectures. Some architectural ideas have computational costs that scale with vocabulary size. Examples include explicit learned token-token interaction matrices ($10K^2 = 100M$ entries vs $32K^2 = 1B$ entries), per-token KV caches with token-specific projection heads, position-token interaction tables, and dense token co-occurrence priors injected as learned biases. These architectures have not been seriously explored in the LLM literature, in part because their cost scales with vocabulary size. Primoji's smaller vocabulary may make such architectures feasible at parameter budgets where they would otherwise be impractical. We have not tested any of these in this work.

Hierarchical parallelism through composition boundaries. Primoji compositions are explicit semantic units with clear boundaries; BPE tokens have no such structure. This enables hierarchical processing architectures: a local encoder pools primitive tokens within each composition into a single concept vector (parallel across compositions), a global transformer processes the concept sequence (sequential but ∼3× fewer positions than the token sequence), and a local decoder expands concepts back to tokens (parallel). The global transformer's attention compute drops by ∼9× at the same context, KV cache shrinks by ∼3×, and effective context window expands by ∼3×. BLT [13] implements similar hierarchical processing with entropy-based patches over byte sequences; Primoji's compositions provide semantic patches by construction. BPE cannot use this architecture because its tokens have no meaningful boundaries.

Mixture of Experts with per-expert dictionaries. A natural application is MoE systems where each expert has its own domain-specific Primoji dictionary while sharing the primitive embeddings. The tokenization itself could become the routing mechanism: if an input contains HEALTH compositions, route to the medical expert. Cross-domain transfer would happen through shared primitive embeddings.

Architecture optimization. Our experiments use BPE-standard architectures, leaving open whether architectures that reallocate the freed embedding parameters to FFN, heads, or MoE experts help. Initial extreme-depth experiments (73 layers $\times$ 1024 hidden at 1B) showed only marginal improvement ($\sim$1.5%), suggesting FFN-heavy and head-heavy variants are the more promising design directions.

Cross-lingual representation. The NSM primes are verified across 30+ language families, making the primitive vocabulary language-agnostic by design. A model trained on English primoji tokens could in principle transfer to other languages by swapping the word-to-primitive dictionary while retaining the primitive embeddings.

Automated dictionary construction. DeepNSM [2] fine-tunes LLMs to generate NSM explications for arbitrary words. Combining DeepNSM with corpus-frequency extraction could automate Primoji dictionary construction for new domains.

6 Related Work

Tokenization. BPE [17], WordPiece [23], Unigram LM [10], and SentencePiece [11] define the standard frequency-driven tokenization landscape. MorphPiece [8] introduced morphologically aware tokenization, with MorphBPE [1] confirming gains at 300M and 1B scales across four languages. BLT [13] is a byte-level transformer matching BPE at 8B scale with dynamic entropy-based patching. Singh and Strouse [18] demonstrated single-digit tokenization improves arithmetic by +22pp. Gallé [4] showed that BPE's compositionality accounts for only a small fraction of its effectiveness. We compare directly only against BPE in this work. Empirical comparison against Unigram LM, WordPiece, BLT, and MorphPiece across the same training setup would strengthen future evaluations.

Concept-level training. Iyer et al. [7] propose concept-level prediction objectives where multiple surface forms are grouped under a single concept, showing that predicting concepts rather than tokens yields lower perplexity and improved robustness to domain shifts. Their approach changes the training objective on standard BPE tokens; Primoji changes the tokens themselves. The two approaches are complementary and provide convergent evidence that concept-level structure improves language model learning.

Semantic primitives. Wierzbicka [20] and Goddard and Wierzbicka [5] established and refined Natural Semantic Metalanguage over five decades. DeepNSM [2] fine-tuned LLMs to generate NSM explications, using NSM as output; complementary to our use of NSM as input vocabulary.

Compositional writing systems. Egyptian hieroglyphic determinatives, Chinese radical composition, and German productive compounding are millennia-old examples of compositional semantic structure supporting productive generalization in human language processing.

Emoji semantics. EmojiNet [22], emoji2vec [3], and ELCo [24] validated emoji as meaningful tokens with compositional human accuracy.

Vocabulary and architecture. Wies et al. [21] showed that when vocabulary size is small relative to model width, additional width provides diminishing returns. Hash Layers [16] validated deterministic routing by token ID. Kaplan et al. [9] demonstrate that LLMs engage in an intrinsic detokenization process, combining BPE subwords into coherent whole-word representations in the early and middle layers, evidence that models expend capacity rebuilding the word-level structure that BPE tokenization discards. Primoji's compositional vocabulary is one way to provide such structure at the input layer. Vocab Diet [15] introduced morphological transformation vectors composed with base-form embeddings for vocabulary compression, the closest prior work to our compositional embedding aliases but operating on morphological rather than semantic decomposition.

Efficient models. BitNet [19, 12] demonstrated 1-bit and 1.58-bit transformer training. Chinchilla-optimal training [6] guides our 1B compute allocation.

7 Conclusion

We presented Primoji, a compositional semantic tokenizer that replaces BPE's frequency-driven vocabulary with semantically structured tokens built from Unicode emoji, NSM primitives, and common word tokens. The central finding is that a 10K-token vocabulary built from semantic primitives can achieve performance comparable to BPE's 32,768-token vocabulary on FineWeb-Edu, at both 125M and 1B parameters.

At 125M parameters, Primoji achieves 5.5% lower BPB than BPE at equal FLOPs (1.080 vs 1.144 at one epoch). At 1B parameters, the two approaches converge to near-parity, with Primoji winning by 2% per document and tying at equal FLOPs. The central empirical contribution is the demonstration that a fundamentally different and 3× smaller vocabulary built from semantic primitives can remain competitive with BPE, not the BPB margin at any particular scale.

We propose that this vocabulary efficiency may enable a research program inaccessible to BPE at the same parameter budget. Architectural ideas whose computational cost scales with vocabulary size become more tractable at 10K than at 32K. Hierarchical processing through composition boundaries could enable substantial reductions in attention compute because Primoji's compositions provide semantic patches by construction. Mixture-of-experts systems could use per-expert dictionaries with shared primitive embeddings, making tokenization itself the routing mechanism. Multilingual transfer becomes a dictionary swap. Domain specialization becomes more tractable via automated NSM explication pipelines. We have not tested any of these directions in this work and present them as motivation for follow-up research.

We also introduce two technical contributions independent of the BPB results. Compositional embedding aliases encode semantic decomposition in the embedding layer (e.g. embedding("is") = mean(BE, NOW)) without expanding sequences, a technique applicable to any tokenizer. A configurable tokenization framework exposes five tunable design dimensions, allowing practitioners to explicitly trade compression for semantic structure. Neither contribution exists in BPE or any other current tokenization approach.

We resolve a byte-preference problem in compositional generation by tracing it to vocabulary gaps rather than architectural limitations. Adequate vocabulary coverage eliminates the problem entirely, reframing what initially appeared to be a fundamental mismatch between compositional tokens and autoregressive training.

Code and Data Availability

Source code for the Primoji tokenizer, the dictionary build pipeline, the training scripts, and the evaluation scripts is available at https://github.com/frane/primoji. All training runs use FineWeb-Edu [14], a publicly available dataset. Total experiment cost: under $200 on consumer and cloud GPUs.

References

[1] Ehsaneddin Asgari, Yassine El Kheir, and Mohammad Ali Sadraei Javaheri. MorphBPE: A Morpho-Aware Tokenizer Bridging Linguistic Complexity for Efficient LLM Training Across Morphologies, February 2025. URL http://arxiv.org/abs/2502.00894. arXiv:2502.00894 [cs].

[2] Raymond Baartmans, Matthew Raffel, Rahul Vikram, Aiden Deringer, and Lizhong Chen. Towards Universal Semantics With Large Language Models, 2025. URL https://arxiv.org/abs/2505.11764. Version Number: 3.

[3] Ben Eisner, Tim Rocktäschel, Isabelle Augenstein, Matko Bosnjak, and Sebastian Riedel. emoji2vec: Learning Emoji Representations from their Description. In *Proceedings of The Fourth International Workshop on Natural Language Processing for Social Media*, pages 48–54, Austin, TX, USA, 2016. Association for Computational Linguistics. doi: 10.18653/v1/W16-6208. URL http://aclweb.org/anthology/W16-6208.

[4] Matthias Gallé. Investigating the Effectiveness of BPE: The Power of Shorter Sequences. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 1375–1381, Hong Kong, China, 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1141. URL https://www.aclweb.org/anthology/D19-1141.

[5] Cliff Goddard and Anna Wierzbicka. *Words and meanings: lexical semantics across domains, languages, and cultures*. Oxford linguistics. Oxford University Press, Oxford New York, NY, first published in paperback edition, 2016. ISBN 978-0-19-966843-4 978-0-19-878355-8.

[6] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, Erich Elsen, Jack W Rae, Oriol Vinyals, and Laurent Sifre. Training Compute-Optimal Large Language Models, 2022. URL https://arxiv.org/abs/2203.15556. Version Number: 1.

[7] Laya Iyer, Pranav Somani, Alice Guo, Dan Jurafsky, and Chen Shani. Beyond Tokens: Concept-Level Training Objectives for LLMs, 2026. URL https://arxiv.org/abs/2601.11791. Version Number: 2.

[8] Haris Jabbar. MorphPiece : A Linguistic Tokenizer for Large Language Models, 2023. URL https://arxiv.org/abs/2307.07262. Version Number: 2.

[9] Guy Kaplan, Matanel Oren, Yuval Reif, and Roy Schwartz. From Tokens to Words: On the Inner Lexicon of LLMs. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://arxiv.org/abs/2410.05864.

[10] Taku Kudo. Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 66–75, Melbourne, Australia, 2018. Association for Computational Linguistics. doi: 10.18653/v1/P18-1007. URL http://aclweb.org/anthology/P18-1007.

[11] Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71, Brussels, Belgium, 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-2012. URL http://aclweb.org/anthology/D18-2012.

[12] Shuming Ma, Hongyu Wang, Lingxiao Ma, Lei Wang, Wenhui Wang, Shaohan Huang, Li Dong Ruiping Wang, Jilong Xue, and Furu Wei. The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits, 2024. Version Number: 1.

[13] Artidoro Pagnoni, Ram Pasunuru, Pedro Rodriguez, John Nguyen, Benjamin Muller, Margaret Li, Chunting Zhou, Lili Yu, Jason Weston, Luke Zettlemoyer, Gargi Ghosh, Mike Lewis, Ari Holtzman, and Srinivasan Iyer. Byte Latent Transformer: Patches Scale Better Than Tokens, 2024. URL https://arxiv.org/abs/2412.09871. Version Number: 1.

[14] Guilherme Penedo, Hynek Kydlíček, Loubna Ben allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale, October 2024. URL http://arxiv.org/abs/2406.17557. arXiv:2406.17557 [cs].

[15] Yuval Reif, Guy Kaplan, and Roy Schwartz. Vocab Diet: Reshaping the Vocabulary of LLMs with Vector Arithmetic, 2025. URL https://arxiv.org/abs/2510.17001. Version Number: 1.

[16] Stephen Roller, Sainbayar Sukhbaatar, Arthur Szlam, and Jason Weston. Hash Layers For Large Sparse Models, 2021. URL https://arxiv.org/abs/2106.04426. Version Number: 3.

[17] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural Machine Translation of Rare Words with Subword Units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany, 2016. Association for Computational Linguistics. doi: 10.18653/v1/P16-1162. URL http://aclweb.org/anthology/P16-1162.

[18] Aaditya K Singh and DJ Strouse. Tokenization counts: the impact of tokenization on arithmetic in frontier LLMs, 2024. URL https://arxiv.org/abs/2402.14903. Version Number: 1.

[19] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. BitNet: Scaling 1-bit Transformers for Large Language Models, 2023. URL https://arxiv.org/abs/2310.11453. Version Number: 1.

[20] Anna Wierzbicka. *Semantic primitives*. Number 22 in Linguistische Forschungen. Athenäum-Verl, Frankfurt a. M, 1972. ISBN 978-3-7610-4822-1.

[21] Noam Wies, Yoav Levine, Daniel Jannai, and Amnon Shashua. Which Transformer architecture fits my data? A vocabulary bottleneck in self-attention, June 2021. URL https://arxiv.org/abs/2105.03928.

[22] Sanjaya Wijeratne, Lakshika Balasuriya, Amit Sheth, and Derek Doran. A semantics-based measure of emoji similarity. In *Proceedings of the International Conference on Web Intelligence*, pages 646–653, Leipzig Germany, August 2017. ACM. ISBN 978-1-4503-4951-2. doi: 10.1145/3106426.3106490. URL https://dl.acm.org/doi/10.1145/3106426.3106490.

[23] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V. Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, Jeff Klingner, Apurva Shah, Melvin Johnson, Xiaobing Liu, Łukasz Kaiser, Stephan Gouws, Yoshikiyo Kato, Taku Kudo, Hideto Kazawa, Keith Stevens, George Kurian, Nishant Patil, Wei Wang, Cliff Young, Jason Smith, Jason Riesa, Alex Rudnick, Oriol Vinyals, Greg Corrado, Macduff Hughes, and Jeffrey Dean. Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation, October 2016. URL http://arxiv.org/abs/1609.08144. arXiv:1609.08144 [cs].

[24] Zi Yun Yang, Ziqing Zhang, and Yisong Miao. The ELCo Dataset: Bridging Emoji and Lexical Composition. In Nicoletta Calzolari, Min-Yen Kan, Veronique Hoste, Alessandro Lenci, Sakriani Sakti, and Nianwen Xue, editors, *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 15899–15909, Torino, Italia, May 2024. ELRA and ICCL. URL https://aclanthology.org/2024.lrec-main.1381/.